Data Structures (ETCCDS202)

Unit–3 Assignment: Sorting Algorithms

Course Details

Name of the School: School of Engineering & Technology

Program (Class / Semester / Batch): B.Tech (CSE /Robotics and AI) Sem-A

Course Title: Data Structures

Course Code: ETCCDS202

Unit Number: 3

Unit Title: Sorting Algorithms

Student Name: Ajitesh Singh Gusain

Roll No. :2501630014

Submitted to: Dr . Aarti Sangwan

Task 3: Reconstruct a Binary Tree from Traversals

1. Tree Reconstruction

3.1 Step-by-Step Reconstruction

Given the traversals Preorder = [A, B, D, E, C, F] and Inorder = [D, B, E, A, C, F]:

1. Identify Root: In Preorder traversal, the first element is always the root. Therefore, Root = A.
2. Split Inorder: Locate 'A' in the Inorder list to separate the left and right subtrees.
    - Left Subtree (Inorder): [D, B, E]
    - Right Subtree (Inorder): [C, F]

3.3 Utility in Systems

Tree reconstruction is a fundamental process in computer systems for several reasons:

- Expression Parsing: Compilers use these techniques to reconstruct Expression Trees from flat code, allowing for the correct evaluation of mathematical operations based on operator precedence.
- Data Recovery: If a hierarchical data structure is serialized (flattened) for storage or transmission, reconstruction algorithms allow the system to perfectly restore the original parent-child relationships.

```python
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

# Task 3: Tree Reconstruction
def build_tree(preorder, inorder):
    if not inorder:
        return None
    root_val = preorder.pop(0)
    root = Node(root_val)
    idx = inorder.index(root_val)
    root.left = build_tree(preorder, inorder[:idx])
    root.right = build_tree(preorder, inorder[idx+1:])
    return root
```

Task 4: Binary Search Tree (BST) Operations + Edge Cases

Task 4: BST Performance and Height Analysis

4.1 The Impact of Insertion Order on Tree Height

The height of a Binary Search Tree (BST) is directly influenced by the sequence in which keys are inserted. As shown in the comparison:

- Balanced Input (Set A): Inserting keys like [50, 30, 70, 20, 40, 60, 80] results in a Balanced Tree. The height is minimized to O(log n), ensuring that search operations remain highly efficient by discarding half the search space at each level.
- Sorted Input (Set B): Inserting keys in a strictly increasing order, such as [10, 20, 30, 40, 50], results in a Right-Skewed Tree. In this state, the tree structurally mimics a Linked List.

4.2 Performance Reflection

When a tree becomes skewed as seen in the diagram:

- Search Complexity: Degrades from O(\log n) to O(n). To find the value 50, the algorithm must now visit every single node in the hierarchy.
- Conclusion: The efficiency of a standard BST is highly volatile. To maintain consistent performance in real-world database indexing or file systems, self-balancing structures (like AVL or Red-Black trees) are used to prevent this linear degradation.

4.3 Summary Table

Tree Type	Height	Search Complexity	Visual Structure
Balanced	O(\log n)	O(\log n)	Wide and shallow
Skewed	O(n)	O(n)	Long and narrow

```python
# Task 4: BST Operations
class BST:
    def __init__(self):
        self.root = None

    def insert(self, key):
        self.root = self._insert(self.root, key)

    def _insert(self, node, key):
        if not node: return Node(key)
        if key < node.val: node.left = self._insert(node.left, key)
        else: node.right = self._insert(node.right, key)
        return node

    def delete(self, key):
        self.root = self._delete(self.root, key)

    def _delete(self, node, key):
        if not node: return None
        if key < node.val: node.left = self._delete(node.left, key)
        elif key > node.val: node.right = self._delete(node.right, key)
        else:
            if not node.left: return node.right
            if not node.right: return node.left
            temp = self._min_node(node.right)
            node.val = temp.val
            node.right = self._delete(node.right, temp.val)
        return node

    def _min_node(self, node):
        while node.left: node = node.left
        return node

    def inorder_list(self, node, res):
        if node:
            self.inorder_list(node.left, res)
            res.append(node.val)
            self.inorder_list(node.right, res)
        return res
```

Task 6: BFS and DFS

6.1. Breadth-First Search (BFS)

Start Node: 0

BFS Order: 0, 1, 2, 3, 4

Step	Action	Queue State (Front to Back)	Visited Nodes
1	Start at 0, enqueue 0	[0]	{0}
2	Dequeue 0, enqueue neighbors 1, 2	[1, 2]	{0, 1, 2}
3	Dequeue 1, enqueue neighbor (2 is visited)	[2]	{0, 1, 2}
4	Dequeue 2, enqueue neighbor 3	[3]	{0, 1, 2, 3}
5	Dequeue 3, enqueue neighbor 4	[4]	{0, 1, 2, 3, 4}
6	Dequeue 4, queue empty	[]	{0, 1, 2, 3, 4}

6.2. Depth-First Search (DFS)

Start Node: 0

DFS Order: 0, 1, 2, 3, 4

The Recursion Stack Concept

DFS is naturally implemented using a recursive approach, which utilizes the system's call stack.

1. When a node is visited, the function calls itself for the first unvisited neighbor.
2. This "pushes" the current state onto the recursion stack and moves deeper into the graph.

3. Once a node has no more unvisited neighbors, the function "returns," which effectively "pops" the node from the stack to backtrack to the previous caller.

6.4. Time Complexity: O(V + E)

The time complexity for both BFS and DFS is O(V + E), where V is the number of vertices and E is the number of edges.

Explanation:

- V (Vertices): Every vertex is visited exactly once. In BFS, each node is enqueued and dequeued once; in DFS, each node is pushed to and popped from the recursion stack once.
- E (Edges): For every vertex visited, the algorithm iterates through its adjacency list to check its neighbours. In an undirected graph, each edge is looked at exactly twice (once from each endpoint).

Because the algorithms perform a constant amount of work for every node and every edge, the total time is proportional to the sum of vertices and edges.

Task 7: Application Mini-Problem (Shortest Path in Unweighted Graph)

7.2. Shortest Path Computation

To find the shortest path from Node 0 to Node 4, we look for the minimum number of edges required to connect them:

1. Start at 0.
2. From 0, we can go to Node 2 (1 edge).
3. From 2, we can go to Node 3 (1 edge).
4. From 3, we can go to Node 4 (1 edge).

Shortest Path: 0 to 2 to 3 to 4

Path Length: 3 edges

(Note: The path 0 to 1to 2 to 3 to 4 is also a valid path, but it has a length of 4 edges, making it longer.)

Why BFS Guarantees the Shortest Path

In an unweighted graph, Breadth-First Search (BFS) is guaranteed to find the shortest path because of its "level-by-level" exploration strategy:

- Layered Expansion: BFS explores all nodes at distance k from the source before moving on to any nodes at distance k+1.
- First Discovery: Because it explores radially outward, the first time a target node is reached, it must be via the fewest number of edges possible.
- Uniform Edge Weights: In unweighted graphs, every edge has an implicit weight of 1. Since BFS never "skips" a level, there is no possibility of finding a "shorter" path later in the search that was missed initially.

If the search were to reach Node 4 at level 3, we can be certain no path exists at level 1 or 2, otherwise, BFS would have discovered it earlier.

```python
# Task 6 & 7: Graph Algorithms
def bfs(adj, start):
    visited, queue, order = {start}, [start], []
    while queue:
        u = queue.pop(0)
        order.append(u)
        for v in adj.get(u, []):
            if v not in visited:
                visited.add(v)
                queue.append(v)
    return order

def shortest_path_unweighted(adj, src, dst):
    queue = [[src]]
    visited = {src}
    while queue:
        path = queue.pop(0)
        node = path[-1]
        if node == dst:
            return path, len(path) - 1
        for neighbor in adj.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(path + [neighbor])
    return None, 0
```

Output

unit4_demo_tests.py

```
PS C:\Users\ajite\OneDrive\Desktop\ASSIGNMENT FOLDER\assignment of second sem\dsa assignment\theory assignment\unit 4> python
second sem\dsa assignment\theory assignment\unit 4\unit4_trees_graphs.py"
PS C:\Users\ajite\OneDrive\Desktop\ASSIGNMENT FOLDER\assignment of second sem\dsa assignment\theory assignment\unit 4> python
second sem\dsa assignment\theory assignment\unit 4\unit4_demo_tests.py"
Initial BST Inorder: [20, 30, 40, 50, 60, 70, 80]
Inorder after deleting 20: [30, 40, 50, 60, 70, 80]
Inorder after deleting 30: [40, 50, 60, 70, 80]
Inorder after deleting 50: [40, 60, 70, 80]

Graph BFS from 0: [0, 1, 2, 3, 4]
Shortest Path 0 to 4: [0, 2, 3, 4] (Distance: 3)
PS C:\Users\ajite\OneDrive\Desktop\ASSIGNMENT FOLDER\assignment of second sem\dsa assignment\theory assignment\unit 4>
```

Github link:-https://github.com/ajiteshgusain/dsa-assignment/tree/main/theory%20assignment/unit%204